

Competitive Programming

Section 72: Master+ Flow And Convex Optimization Mixed Contest

Competitive Programming Course

Table of contents

1	Contest contract	3
2	Suggested contest use	4
3	Reduction triage	5
4	How to find A: conservation before optimization	6
5	How to find E: price the one awkward statistic	7
6	How to find F: expose discrete derivatives	8
7	Proof map	9
8	C++ and Python reliability	10
9	Stress-oracle design	11
10	Exercises	12

1 Contest contract

This six-problem contest mixes Sections 70–71. All statements are local and self-contained. The order is progressive, but later tasks deliberately hide the reduction behind assignment, graph, or resource language.

Problem	Main compression
A	lower bounds become vertex imbalances
B	exact flow plus lower-bound imbalances
C	choose exactly K workers while satisfying project quotas
D	remove mandatory lower allocation, then select marginals
E	Lagrangian price on a fixed-size spanning forest
F	quota flow plus convex marginal-cost edges

2 Suggested contest use

Allow four hours. Before coding, write one sentence naming the conserved quantity or ordered marginal structure. For E and F, also write how ties or the forcing constant are certified. Upsolve every task with a tiny brute-force oracle; these techniques are especially vulnerable to plausible but false binary-search conditions.

3 Reduction triage

Before building a graph or writing a heap, isolate which part of the statement is difficult.

Difficult phrase	First transformation to investigate
every choice has a minimum and maximum usage	subtract lower bounds
exactly F indistinguishable units, additive edge costs	min-cost flow
one worker chooses one eligible project	bipartite unit flow
later units become progressively more expensive	marginal-cost expansion
an exact count breaks an otherwise easy optimizer	Lagrangian price
values must be monotone under absolute loss	convex derivative / slope trick

Several problems contain two rows of this table. Apply transformations one at a time and preserve their proof obligations. For example, F first turns quadratic congestion into ordered marginal slots, then embeds those slots in a quota-constrained assignment flow.

4 How to find A: conservation before optimization

Ignore all costs because A has none. Exact group demand means fixed flow out of the source; room minima and maxima are bounded edges to the sink. The allowed group-room pairs form the only unit middle edges. A fixed return edge closes the network into a circulation.

The important reasoning is bidirectional. A valid seating selection sends one unit along every chosen pair. Conversely, integrality makes every used middle edge a discrete selected pair. Without this second direction, a network may silently admit fractional or repeated choices that the statement forbids.

5 How to find E : price the one awkward statistic

Without the exact number of red edges, E is a minimum fixed-size forest. This tells us what not to replace: keep Kruskal, stop after `n-components` edges, and move only the exact red count into the objective. Adding `lambda` per red edge creates a family of ordinary Kruskal instances.

The binary-search predicate is not “the returned tree has exactly `k`.” At a tie, equally optimal trees can have different counts. The correct observation is an interval: red-first and blue-first Kruskal give the maximum and minimum counts among modified-optimal trees. The target is feasible at that price when it lies between them.

6 How to find F: expose discrete derivatives

A team used x times pays $a \cdot x^2$. Compute consecutive differences:

$$\text{cost}(x) - \text{cost}(x-1) = a \cdot (2x-1).$$

These marginals are nondecreasing, so a minimum-cost flow using x team slots will take the first x . The nonlinear objective has become ordinary parallel unit edges. This “differentiate a discrete convex cost” move is useful whenever the total cost is separable by resource.

7 Proof map

For A–C and F, use three layers:

1. prove the original objects and integral flows correspond;
2. prove capacities, bounds, or rewarded slots encode feasibility exactly;
3. prove the residual algorithm optimizes cost among feasible flows.

For D, prove that the marginal costs are ordered and that threshold counting selects exactly the globally cheapest remaining units. For E, prove monotonicity with respect to the price and tie-interval attainability. A binary search without both facts is only an experiment.

8 C++ and Python reliability

- Use 64-bit capacities only when the total flow can exceed `int`.
- Use 128-bit arithmetic for `BIG`, quadratic marginal sums, and penalized MST values when the product bounds require it.
- In Python, heap records and residual edges dominate memory; avoid one object wrapper per tiny record.
- Update potentials only for vertices reached by Dijkstra.
- Check saturation of every lexicographically rewarded lower-quota slot rather than assuming a large constant worked.

9 Stress-oracle design

For tiny instances, enumerate subsets for A, all integral path flows for B, worker assignments for C and F, bounded repaired sequences for D, and spanning trees for E. The oracle should implement the statement directly. Reusing the same reduction as the optimized solution confirms only that both programs made the same modeling mistake.

Useful adversarial families include equal-cost routes, tight lower bounds, disconnected eligibility graphs, many equal modified MST weights, and several marginals equal to the threshold.

10 Exercises

Solve A–F as one contest. The mixed set contains exactly six new local problems and covers every principal invariant introduced in Sections 70–71.